Relatório de Laboratório

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab03
Grupo (trio)	Amanda Bicalho · Guilherme de Almeida · Pedro Duarte
Link do repositório / GitHub Projects	https://github.com/guilhermeas04/Laboratorio-de-experimentacao-de-software
Data de entrega	27/08/2026

1. Introdução

As métricas DORA são amplamente utilizadas para avaliar o desempenho da entrega de software, principalmente a partir de duas dimensões complementares: velocidade e estabilidade. Essas métricas ajudam equipes e organizações a entender com que frequência entregam novas versões, quanto tempo uma mudança leva para chegar aos usuários, com que frequência as entregas apresentam problemas e quanto tempo é necessário para recuperar o sistema após uma falha. Por esse motivo, tornaram-se uma referência importante para a avaliação de práticas de desenvolvimento e entrega contínua na Engenharia de Software. Entretanto, aplicar essas métricas a repositórios open source apresenta algumas limitações. O GitHub, por exemplo, não registra necessariamente todos os deploys realizados em produção nem permite identificar de forma direta quais falhas chegaram aos usuários finais. Por isso, neste laboratório utilizamos releases publicadas como uma aproximação para deploys e execuções do GitHub Actions como representação da atividade de integração e entrega contínua. O objetivo é analisar dados públicos de repositórios reais ao longo de uma janela de doze meses, investigando não apenas os valores das métricas DORA, mas também as limitações e a confiabilidade das aproximações utilizadas.

RQ1. Qual é a frequência de deploys dos repositórios populares que usam CI/CD? **Hipótese informal:** esperamos que a frequência de deploy varie significativamente entre os repositórios e seja baixa ou moderada na maior parte da amostra, devido às diferenças entre bibliotecas, ferramentas e aplicações.

RQ2. Qual é o tempo entre um commit e seu respectivo deploy? **Hipótese informal:** esperamos que o lead time apresente uma distribuição assimétrica, com grande parte dos commits chegando a uma release em poucos dias, enquanto alguns commits mais antigos aumentam o tempo observado em determinadas releases.

RQ3. Qual é a taxa de falha das mudanças entregues por esses repositórios? **Hipótese informal:** esperamos encontrar diferenças entre a taxa de falha calculada a partir de execuções de CI e aquela inferida por releases corretivas, uma vez que uma falha em um pipeline não representa necessariamente uma falha em produção.

RQ4. Qual é o tempo de recuperação após uma execução de CI/CD com falha? **Hipótese informal:** esperamos que a maioria dos episódios de falha seja resolvida em poucas horas, embora alguns casos apresentem períodos de recuperação mais longos ou não possam ser completamente observados dentro da janela analisada.

RQ5. Repositórios com maior frequência de deploy apresentam maior ou menor taxa de falha? **Hipótese informal:** esperamos encontrar uma associação negativa ou próxima de zero entre frequência de deploy e taxa de falha, considerando que uma maior velocidade de entrega não implica necessariamente menor estabilidade.

RQ6. Quais características dos repositórios estão associadas a um melhor desempenho DORA? **Hipótese informal:** esperamos que características como linguagem principal, popularidade, idade do repositório e tamanho da comunidade apresentem associação com algumas métricas DORA, mas que nenhum fator isolado seja suficiente para explicar todas as diferenças de desempenho observadas.

RQ7. O quanto a classificação DORA de um repositório depende da definição operacional escolhida? **Hipótese informal:** esperamos que uma parcela relevante dos repositórios mude de classificação quando forem consideradas diferentes formas de cálculo, como inclusão de pré-releases, uso de tags

ou variações na definição de lead time e taxa de falha.

2. Contexto

Este estudo corresponde ao Laboratório 03 da disciplina de Laboratório de Experimentação de Software, etapa em que o grupo transforma conceitos de medição de software em uma coleta automatizada e reproduzível. Ele sucede os laboratórios anteriores no percurso acadêmico, mas possui escopo e dados próprios: não reutiliza os trials do LAB2 nem depende do Kanban ou de artefatos de outro experimento. A conexão com os trabalhos anteriores está no processo experimental, que mantém a prática de formular questões, definir variáveis, automatizar a coleta, registrar decisões e preservar evidências para análise e replicação. O objeto de estudo são repositórios públicos e populares do GitHub que utilizam GitHub Actions. Para cada repositório, observamos releases publicadas, tags, commits entre releases e execuções de workflows no branch padrão durante uma janela de doze meses. Esses dados permitem estimar as quatro dimensões centrais de desempenho de entrega: frequência de deploy, lead time for changes, change failure rate e tempo de recuperação. Como o GitHub não informa diretamente todos os deploys em produção, releases e execuções de CI são tratados como proxies e suas limitações são registradas na análise. A base conceitual das métricas é o modelo DORA apresentado por Forsgren, Humble e Kim em *Accelerate: The Science of Lean Software and DevOps* (2018), complementado pelas definições atuais do projeto DORA. O planejamento das perguntas, métricas e procedimentos também segue a lógica Goal–Question–Metric (GQM), proposta por Basili, Caldiera e Rombach, que relaciona objetivos de medição a questões observáveis e métricas operacionais. Essa combinação orienta a passagem do objetivo geral de avaliar desempenho de entrega para as RQs e os indicadores calculados neste laboratório.

3. Metodologia

3.1 Principais Desafios

A execução do Lab03 apresentou desafios técnicos e metodológicos relacionados principalmente à coleta e à padronização de dados provenientes da API do GitHub. O primeiro deles foi lidar com a paginação e os limites de requisições da API, uma vez que a amostra final continha 300 repositórios e cada repositório exigia consultas de releases, tags, commits e GitHub Actions. Para reduzir o risco de interrupções e evitar consultas repetidas, foi implementado um cliente próprio para a API, com autenticação por token, cache local das respostas, tentativas automáticas em caso de falhas temporárias e tratamento de limite de taxa.

Outro desafio foi definir operacionalmente as métricas DORA em situações nas quais os dados disponíveis eram incompletos ou ambíguos. O lead time, por exemplo, foi calculado em duas variantes: a partir da publicação de releases e a partir da autoria dos commits associados. Da mesma forma, o Change Failure Rate foi calculado usando tanto a evidência de falha em workflows quanto a heurística corretiva baseada na relação entre releases subsequentes. Essas decisões foram necessárias porque nem todos os repositórios possuíam releases regulares, workflows ativos ou informações suficientes para determinar diretamente se uma mudança havia causado falha. Também foi necessário tratar

epositórios sem GitHub Actions habilitado, com poucas releases, tags sem data de commit recuperável ou histórico insuficiente para o cálculo de determinadas métricas. Nesses casos, os registros não foram simplesmente convertidos em zero: foram marcados como ausentes ou excluídos da métrica específica, preservando a diferença entre ausência de evidência e desempenho nulo. Essa decisão evitou distorções nas estatísticas e tornou explícitos os limites da base coletada.

Por fim, a validação manual da heurística de releases exigiu a construção de uma amostra-ouro de 60 repositórios, rotulada independentemente pelos integrantes do grupo. Foi necessário resolver divergências entre os rótulos, calcular a concordância por meio do kappa de Fleiss e medir precisão, recall e F1 da heurística. Esse processo evidenciou que a definição de uma "release válida" depende de sinais combinados — tags, datas, commits e workflows — e não apenas da existência de um evento isolado na API.

3.2 Tomadas de Decisão

A linguagem escolhida para o desenvolvimento do pipeline foi Python, por oferecer bibliotecas adequadas para manipulação de arquivos CSV e JSON, análise estatística e geração de visualizações. Para as consultas ao GitHub, foi utilizado um cliente próprio baseado na biblioteca padrão, em vez de uma biblioteca especializada de terceiros. Essa decisão manteve o controle sobre autenticação, paginação, cache, tentativas de nova requisição e tratamento de erros, embora tenha exigido mais código de infraestrutura. A amostra foi definida a partir de uma seleção sistemática de repositórios públicos do GitHub. Foram aplicados filtros de disponibilidade dos dados, atividade no período observado, existência de histórico suficiente e possibilidade de calcular as métricas necessárias. A meta foi obter uma amostra final de pelo menos 300 repositórios após os filtros, evitando que a análise dependesse apenas dos projetos mais conhecidos ou de um único ecossistema de linguagem.

O período de observação foi fixado de janeiro a dezembro de 2025 para todos os repositórios. Essa padronização permitiu comparar frequências de deploy, releases, tags, lead time e falhas dentro da mesma janela temporal. O trade-off foi excluir eventos fora desse intervalo, mesmo quando poderiam fornecer mais histórico, pois misturar períodos diferentes comprometeria a comparabilidade entre os repositórios. Para o lead time, foram mantidas duas definições operacionais. A primeira considera o intervalo entre a autoria do commit e a publicação da release; a segunda utiliza o intervalo entre releases sucessivas. A primeira representa melhor o tempo associado à mudança de código, enquanto a segunda é mais diretamente observável quando os commits ou workflows não possuem informações completas. Em vez de escolher arbitrariamente apenas uma definição, o grupo calculou ambas e avaliou a sensibilidade das classificações DORA às duas alternativas.

O Change Failure Rate também foi calculado por duas abordagens. A primeira utiliza falhas observadas nos workflows do GitHub Actions. A segunda aplica uma heurística corretiva baseada nos eventos posteriores à release, permitindo identificar possíveis falhas mesmo quando o workflow não estava disponível. A heurística foi validada por uma amostra-ouro rotulada manualmente pelos integrantes, com cálculo de precisão, recall, F1 e concordância entre os avaliadores. Quando uma informação necessária não estava disponível, o grupo não a substituiu automaticamente por zero. O registro foi marcado como ausente ou excluído apenas da métrica afetada. Essa decisão evita interpretar "não há dados" como "não houve falha" ou "o desempenho foi perfeito", reduzindo o risco de viés nos

resultados. Como consequência, algumas comparações possuem tamanhos amostrais diferentes, mas essa limitação fica explicitamente documentada.

Por fim, os dados brutos foram mantidos em JSON e cache local, enquanto os resultados consolidados foram exportados para CSV, acompanhados de um dicionário de dados e manifestos de execução. Essa separação preserva a rastreabilidade das respostas originais da API sem tornar o dataset final difícil de consultar. As análises estatísticas e os gráficos foram gerados a partir dos arquivos consolidados, garantindo reprodutibilidade do processamento.acúmulo individual; ao ser atingido, uma tarefa deve avançar para Review ou Done antes que outra seja iniciada.

3.3 Etapas

O desenvolvimento do LAB3 foi organizado em três sprints, acompanhadas pelo GitHub Projects e vinculadas a Issues, commits e Pull Requests. A Sprint 1 foi dedicada à construção da infraestrutura de mineração. Guilherme ficou responsável pela seleção dos repositórios e metadados, pelo funil inicial e pela integração do pipeline; Pedro implementou a coleta de releases, tags, commits, deployment frequency e lead time; Amanda implementou a coleta de workflows, cache, controle de limite da API, CFR baseado em CI e tempo de recuperação. Ao final, a Issue #101 integrou o pipeline, os testes automatizados e a documentação inicial. Na Sprint 2 foi realizada a validação e consolidação dos dados. Guilherme produziu sua rotulação independente da amostra-ouro e calculou a concordância, o kappa de Fleiss e o consenso (#104 e #107). Amanda realizou sua rotulação independente e implementou/refinou a heurística corretiva de releases para o CFR(b), validando-a com precisão, recall e F1 (#105 e #108). Pedro realizou sua rotulação independente e ficou responsável pela coleta final, consolidação do dataset e dicionário de dados (#106 e #109). Essa sprint resultou na amostra final, nas métricas calculadas e nos arquivos tabulares utilizados na análise. A Sprint 3 foi dedicada à análise dos resultados e à preparação das evidências para o relatório. Amanda conduziu as análises de RQ01–RQ04 e a classificação DORA C1 (#115, #118 e #119); Pedro conduziu as correlações de RQ05 e as comparações entre subgrupos da RQ06 (#116, #120 e #121); Guilherme conduziu a análise de sensibilidade da RQ07 e a integração/reprodutibilidade das análises (#117, #122 e #123). Por fim, os resultados estatísticos, tabelas, classificações, gráficos e limitações foram consolidados neste relatório final.

Sprint	Entrega	Responsável	Issue
LAB3S01	Seleção de repositórios e metadados	Guilherme de Almeida Santos	# 95
LAB3S01	Funil de seleção da amostra	Guilherme de Almeida Santos	#96

LAB3S01	Coleta de releases, tags e commits	Pedro Rodrigues Duarte	#97
LAB3S01	Deployment frequency e lead time	Pedro Rodrigues Duarte	#98
LAB3S01	Workflow runs do default branch	Amanda Bicalho Silva	#99
LAB3S01	Cache, rate limit, CFR de CI e recuperação	Amanda Bicalho Silva	#100
LAB3S01	Integração do pipeline, CI e documentação inicial	Guilherme de Almeida Santos	#101
LAB3S02	Rotulação independente da amostra-ouro	Guilherme de Almeida Santos	#104
LAB3S02	Rotulação independente da amostra-ouro	Amanda Bicalho Silva	#105
LAB3S02	Rotulação independente da amostra-ouro	Pedro Rodrigues Duarte	#106
LAB3S02	Fleiss kappa e consenso	Guilherme de Almeida Santos	#107
LAB3S02	Heurística corretiva e validação do CFR(b)	Amanda Bicalho Silva	#108
LAB3S02	Coleta final, dataset e dicionário de dados	Pedro Rodrigues Duarte	#109
LAB3S03	RQ01–RQ02: frequência de deploy e lead time	Amanda Bicalho Silva	#118
LAB3S03	RQ03–RQ04 e classificação DORA C1	Amanda Bicalho Silva	#119
LAB3S03	RQ05: correlação entre métricas DORA	Pedro Rodrigues Duarte	#120

LAB3S03	RQ06: comparação entre subgrupos	Pedro Rodrigues Duarte	#121
LAB3S03	RQ07: análise de sensibilidade	Guilherme de Almeida Santos	#122
LAB3S03	Integração e reprodutibilidade das análises	Guilherme de Almeida Santos	#123

O grupo utilizou um quadro GitHub Projects vinculado ao repositório do LAB3. O fluxo foi composto pelas colunas Backlog → To Do → Doing → Review → Done. As atividades planejadas foram mantidas no Backlog; as selecionadas para a sprint foram movidas para To Do; tarefas iniciadas foram colocadas em Doing; entregas aguardando conferência ficaram em Review; e somente atividades implementadas, testadas e revisadas foram movidas para Done.

A coluna Doing utilizou limite de WIP igual a 2 Issues simultâneas. Quando o limite era atingido, uma atividade precisava avançar para Review ou Done antes do início de outra. Essa política manteve o foco dos integrantes, reduziu tarefas parciais e facilitou a revisão e a integração das alterações.

3.4 Ferramentas

A coleta, o processamento e a validação dos dados foram realizados por scripts próprios desenvolvidos pelo grupo. Nenhuma biblioteca específica para consulta à API do GitHub foi utilizada. As ferramentas empregadas estão descritas a seguir.

Ferramenta	Versão ou tecnologia	Utilização
Python	3.13.7	Implementação dos scripts de coleta, normalização, validação, análise e exportação
GitHub GraphQL API	GraphQL v4	Consulta paginada dos 1.000 repositórios e exportação dos itens do GitHub Projects
Requests	Biblioteca Python	Envio direto das requisições HTTP para o endpoint GraphQL do GitHub
python-dotenv	Biblioteca Python	Leitura segura das variáveis de ambiente armazenadas no arquivo .env
csv	Biblioteca padrão do Python	Exportação e leitura dos dados consolidados e snapshots
statistics	Biblioteca padrão do Python	Cálculo de média, mediana, quartis e limites utilizados na identificação de outliers
Datetime	Biblioteca padrão do Python	Cálculo da idade dos repositórios e dos dias desde a última atualização ou push
Git	Sistema de controle de versão	Versionamento dos scripts, relatórios, dados e snapshots
Github	Plataforma de hospedagem	Armazenamento do repositório, revisão por Pull Requests e rastreabilidade entre

		Issues e commits
Github Projects v2	Ferramenta de gestão	Organização das atividades em Kanban e acompanhamento de Assignees e Status
Markdown	Formato textual	Produção da documentação, relatórios e registros das validações
CSV	Formato de dados	Armazenamento dos 1.000 repositórios coletados e dos snapshots do Project
GitHub Octoverse 2024	Fonte de referência	Definição do conjunto de linguagens consideradas mais populares na RQ05
Matplotlib	Biblioteca Python	Geração dos histogramas, boxplots e gráficos de dispersão das RQ01 a RQ07
Seaborn	Biblioteca Python	Aplicação de estilo comum e construção de visualizações estatísticas
Backend Agg	Backend do Matplotlib	Geração automática de figuras PNG sem interface gráfica
PNG	Formato de imagem	Armazenamento reproduzível das visualizações geradas pelo pipeline

A consulta dos repositórios foi realizada diretamente no endpoint https://api.github.com/graphql. O cliente HTTP foi implementado pelo próprio grupo com autenticação por token, tratamento de erros, retentativas e tempo máximo de resposta. A paginação utiliza os campos pageInfo, endCursor e hasNextPage, coletando os repositórios em páginas de 10 itens para reduzir erros de gateway provocados pelo custo da consulta agregada.

A coleta, a normalização e a validação dos dados foram implementadas em Python, utilizando bibliotecas da distribuição padrão, além de requests e python-dotenv. As visualizações foram produzidas com Matplotlib e Seaborn, configuradas com o backend não interativo Agg, permitindo a geração automática das figuras sem abertura de interface gráfica ou edição manual. Não foram utilizadas bibliotecas específicas para acesso à API do GitHub; as requisições GraphQL foram enviadas diretamente ao endpoint oficial. Os dados consolidados foram armazenados em CSV e processados pelos scripts desenvolvidos pelo próprio grupo.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Deployment frequency	número de releases publicadas na janela ÷ número de semanas da janela	Releases/semana	GitHub REST API — /repos/{owner}/{repo}/releases; Python
RQ02	Lead time por release	Para cada release R: data_publicação(R) − data_do_commit_mais_antigo_incluído(R); resultado do	Dias	GitHub REST API — releases e /compare/{base}...{head}; Python

		repositório = mediana dos valores por release		
RQ02	Lead time por commit	Para cada commit c de uma release: data_publicação(R) − data_autoria(c); resultado do repositório = mediana de todos os commits das releases	Dias	GitHub REST API — releases e commits; Python
RQ03	CFR — proxy de CI	runs com conclusão failure ÷ (runs failure + runs success)	Proporção	GitHub REST API — /repos/{owner}/{repo}/actions/runs; Python
RQ03	CFR — proxy de entrega	Uma release é considerada falha quando é seguida, em até 7 dias, por uma release corretiva. CFR = releases falhas ÷ releases avaliadas; releases nos últimos 7 dias da janela são censuradas	Proporção	Releases, tags e commits do GitHub; heurística corretiva implementada em Python
RQ04	Tempo de recuperação	Para cada episódio: updated_at da primeira execução success posterior − run_started_at da primeira failure; resultado = mediana dos episódios	Horas	GitHub Actions — workflow runs; Python
RQ04	Censura de recuperação	episódios que não terminaram com sucesso na janela ÷ total de episódios iniciados	Proporção	GitHub Actions — workflow runs; Python
RQ05	Correlação entre frequência e falha	Coeficiente ρ de Spearman entre deployment frequency (RQ01) e CFR (RQ03), calculado separadamente para CFR(a) e CFR(b), acompanhado de p-valor e tamanho da amostra	ρ, p-valor e n	scipy.stats.spearmanr; dataset consolidado em CSV
RQ06	Diferenças entre subgrupos	Para cada fator e métrica DORA, comparação das distribuições entre grupos definidos por linguagem, quartis de estrelas, quartis de contribuidores, quartis de idade ou tipo de projeto	Estatística do teste e p-valor	scipy.stats.kruskal ou scipy.stats.mannwhitneyu
RQ06	Correção de múltiplos testes	Ajuste dos p-valores dos testes da RQ06 pelo procedimento de Holm	p-valor ajustado	statsmodels.stats.multitest.multipletests(method="holm")
RQ06	Tamanho de efeito	Para mais de dois grupos: $\varepsilon^2 = H \div (n - 1)$. Para dois grupos: $\delta = 2U \div (n_1 \times n_2) - 1$ (Cliff's	ε^2 ou δ	Python, SciPy e implementação complementar

		delta)		
RQ07	Mudança de classificação DORA	Para cada par de variantes: número de repositórios com categoria diferente ÷ número de repositórios comparáveis	Proporção	Python; classificações calculadas a partir do dataset
RQ07	Classificação DORA	Cada dimensão recebe uma categoria conforme os limites DORA; a categoria final é a mediana inteira inferior das quatro dimensões: frequência, lead time, CFR e recuperação	Elite/High/Medium/Low	Python; regras DORA definidas no projeto
RQ07	Concordância entre variantes	Kappa de Cohen ponderado linear entre as categorias ordinais Elite, High, Medium e Low	Kappa	Implementação Python; matrizes de transição exportadas em CSV
RQ07	Variantes de sensibilidade	C1: release + lead time por release + CFR(a); C2: release + lead time por commit + CFR(b); C3: tag + lead time por commit + CFR(a); C4: tag + lead time por commit + CFR(b)	Categorias e taxa de mudança	GitHub REST API, dataset S02 e s02_tag_metrics.csv

3.6 Inovações Propostas pelo Grupo (30% da nota)

O grupo propôs quatro contribuições adicionais ao escopo mínimo do laboratório. A primeira foi uma arquitetura de coleta resiliente, baseada em um cliente próprio para a API REST do GitHub, com autenticação por token, paginação automática, cache local, tratamento de falhas temporárias e controle do rate limit. Essa decisão permitiu interromper e retomar a coleta sem perder as respostas já obtidas e reduziu a quantidade de requisições repetidas. Os arquivos brutos permaneceram em JSON e cache local, enquanto os resultados consolidados foram versionados em CSV. O impacto dessa escolha será discutido nos Resultados e na Discussão por meio da taxa de reuso do cache, da quantidade de repositórios processados e dos registros de falhas da coleta.

A segunda inovação foi a ampliação da análise de sensibilidade da RQ07. Além das três combinações mínimas previstas no enunciado, o grupo implementou quatro variantes: C1, baseada em releases, lead time por release e CFR(a); C2, baseada em releases, lead time por commit e CFR(b); C3, baseada em tags, lead time por commit e CFR(a); e C4, baseada em tags, lead time por commit e CFR(b). Para cada par de variantes foram calculadas a taxa de mudança de categoria, as matrizes de transição e o kappa de Cohen ponderado. Essa extensão permite verificar se as conclusões dependem simultaneamente da unidade de entrega, da definição de lead time e do proxy de falha utilizado. Os resultados aparecem nos arquivos de classificação, comparações e matrizes de transição da RQ07 e serão interpretados na seção de Discussão.

A terceira contribuição foi a separação explícita entre ausência de evidência e valor zero. Quando um repositório não possuía workflows, tags, releases ou episódios suficientes para uma métrica, o caso era marcado como ausente ou censurado, em vez de ser convertido automaticamente em zero. Além

disso, o grupo reportou a proporção de episódios de recuperação censurados e a quantidade de releases não avaliáveis no cálculo do CFR(b). Essa decisão reduz o risco de interpretar falta de dados como bom desempenho e funciona como um controle adicional de ameaça à validade. Seus efeitos serão apresentados na análise das limitações, na Discussão e na Conclusão.

Por fim, o grupo adotou uma estratégia de reprodutibilidade superior à exigida, com ponto de entrada único do pipeline, testes automatizados, manifestos de execução, dicionário de dados e relatórios gerados automaticamente em CSV, JSON, Markdown e SVG. Dessa forma, outro grupo pode reproduzir a coleta, identificar a janela de observação, verificar a quantidade de repositórios processados e regenerar as análises. Essa contribuição será retomada na Discussão como benefício metodológico e na Conclusão como parte do diferencial do projeto.

4. Resultados

4.1 Coleta de Dados

A coleta final foi executada em outubro de 2026 e produziu uma base com 300 repositórios elegíveis do GitHub, após a aplicação dos filtros definidos na Sprint 2. Ao todo, foram analisados 5.924 repositórios candidatos. A consolidação identificou cinco ocorrências duplicadas durante o processo de seleção; após a remoção dessas ocorrências, a amostra final ficou sem duplicidades e atingiu exatamente o mínimo de 300 repositórios exigido pelo laboratório.

Verificação	Resultado
Repositórios coletados	5.924
Repositórios na amostra final	300
Repositórios elegíveis	300
Repositórios duplicados removidos na consolidação	5
Repositórios com releases válidas	300
Repositórios com GitHub Actions disponível	300
Releases coletadas	5.644
Workflow runs válidos coletados	78.334
Janela de observação	01/01/2025 a 31/12/2025
Dataset consolidado	data/processed/datasets/s02_metrics.csv

A coleta utilizou a API REST do GitHub com paginação, autenticação por token, cache local e tratamento de falhas temporárias. Para cada repositório foram coletados releases, tags, commits entre releases, metadados e execuções de workflows do branch padrão. Todos os 300 repositórios finais possuíam pelo menos um workflow run válido e permaneceram na base para as análises de deployment frequency, lead time, CFR e tempo de recuperação.

Foram identificadas situações de censura e ausência parcial de informação. No cálculo do CFR baseado em releases, 125 releases ficaram censuradas por terem sido publicadas nos últimos sete dias da janela de observação; por isso, não foram incluídas no denominador até que houvesse tempo suficiente para observar uma possível release corretiva. Também foram identificados 24 episódios de

recuperação censurados, correspondentes a falhas que não apresentaram execução bem-sucedida posterior dentro da janela. Esses casos foram mantidos e contabilizados separadamente, em vez de serem convertidos em zero ou removidos.

As distribuições apresentaram valores extremos principalmente na quantidade de workflow runs, releases e tempo de recuperação. O maior número observado foi de 3.133 workflow runs em um único repositório, enquanto o menor foi de 50. Esses valores foram mantidos por representarem diferenças reais de atividade entre os projetos. Para reduzir a influência dos extremos, os resultados foram resumidos prioritariamente por mediana e intervalo interquartil, conforme definido no enunciado.

Os dados brutos permaneceram em arquivos JSON e no cache local para permitir auditoria e reexecução da coleta. Os resultados tabulares foram consolidados em CSV, acompanhados do manifesto de execução e do dicionário de dados. Dessa forma, a base final preserva tanto os registros válidos quanto as limitações de observação identificadas durante a mineração.

4.2 Visualização Gráfica

4.2.1 RQ01 — Qual a frequência de deploys dos repositórios populares que usam CI/CD?

Métrica: deployment frequency, em releases por semana.

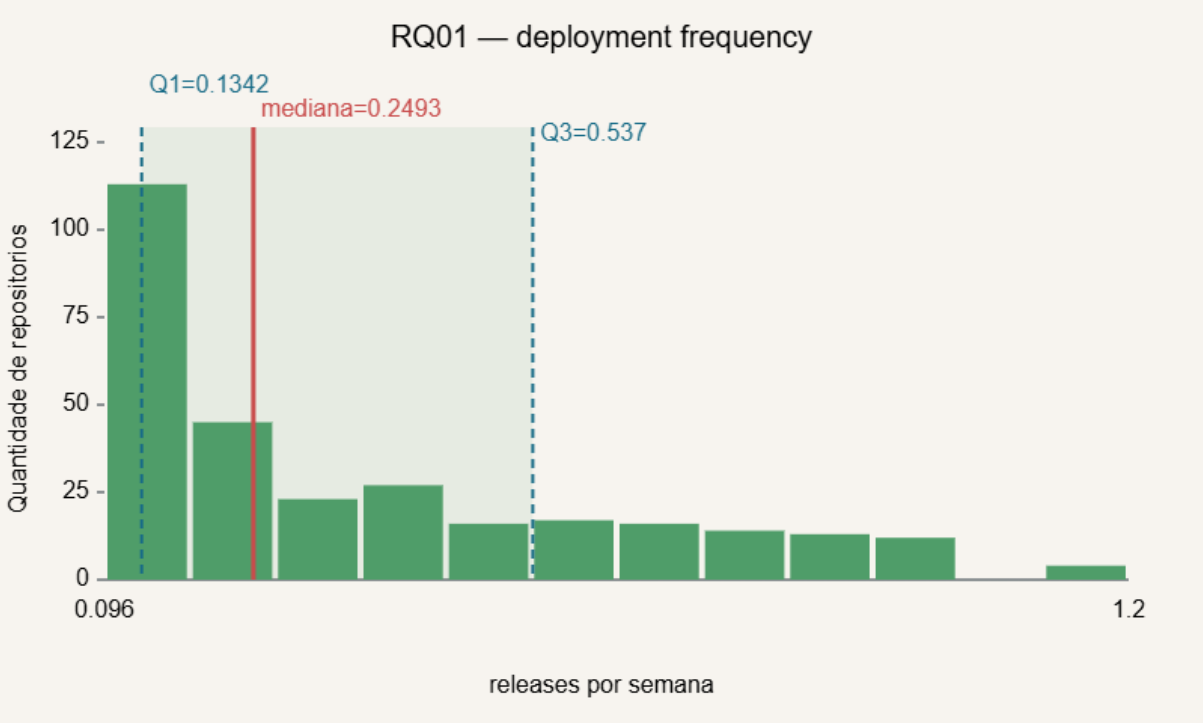


Figura 1 (histograma) — Distribuição da idade dos repositórios analisados. Fonte: Elaboração própria.

O histograma apresenta a distribuição da frequência de releases por semana entre os 300 repositórios analisados. O eixo x representa a deployment frequency, em releases por semana, e o eixo y representa a quantidade de repositórios. A linha vermelha indica a mediana, enquanto as linhas azuis tracejadas e a faixa sombreada representam o primeiro e o terceiro quartis.

A mediana foi de 0,2493 releases por semana, com primeiro quartil de 0,1342 e terceiro quartil de 0,5370, resultando em um IQR de 0,4027 releases por semana. Isso significa que metade dos

repositórios está entre aproximadamente 0,13 e 0,54 releases semanais. A distribuição é assimétrica, com maior concentração nas frequências mais baixas e poucos repositórios apresentando cadência significativamente superior.

Essa visualização responde diretamente à RQ01 porque mostra a distribuição numérica da frequência de deploys, e não apenas a classificação dos repositórios nas categorias DORA. Como a métrica utiliza releases publicadas, ela deve ser interpretada como um proxy de deployment frequency, não como uma observação direta de deploy em produção.

4.2.2 RQ02 — Qual o tempo entre um commit e seu respectivo deploy?

Métrica: lead time for changes, em duas variantes obrigatórias

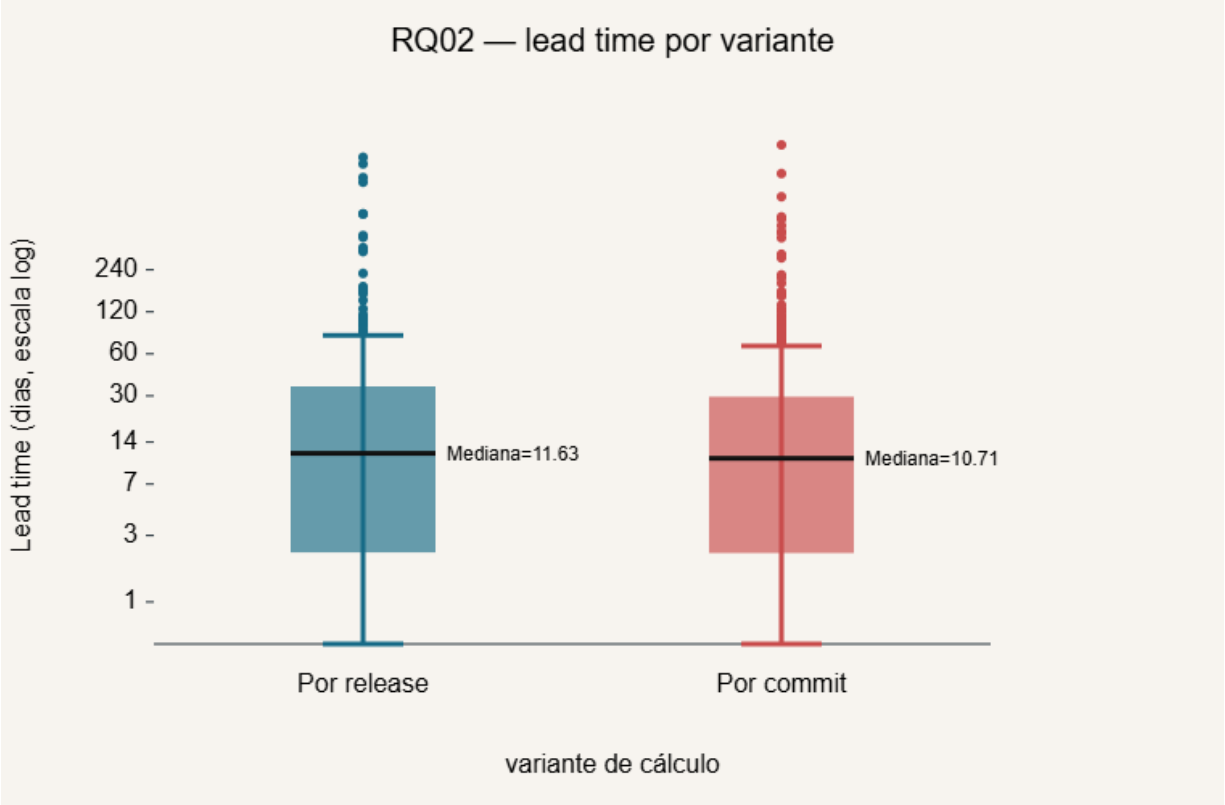


Figura 2 (Boxplot) — Comparação do lead time por release e por commit.

O boxplot compara as duas definições obrigatórias de lead time. O eixo x apresenta as variantes por release e por commit, enquanto o eixo y representa o tempo entre a autoria do commit e a publicação da release, em dias, utilizando escala logarítmica para facilitar a visualização dos valores extremos. A linha central de cada caixa representa a mediana, os limites da caixa representam o primeiro e o terceiro quartis, e os pontos externos indicam possíveis valores extremos.

Na variante por release, a mediana foi de 11,6280 dias, com primeiro quartil de 2,2709 dias e terceiro quartil de 35,1451 dias, resultando em IQR de 32,8741 dias. Na variante por commit, a mediana foi de 10,7128 dias, com primeiro quartil de 2,2464 dias e terceiro quartil de 29,6179 dias, resultando em IQR de 27,3715 dias.

As medianas próximas indicam que as duas definições produzem resultados centrais semelhantes. Entretanto, a maior dispersão da variante por release mostra que releases contendo commits antigos

podem aumentar significativamente o lead time calculado. A variante por commit reduz a influência desses commits isolados ao considerar individualmente todos os commits incluídos nas releases. Eles foram calculados para os 300 repositórios válidos, sem valores ausentes.

4.2.3 RQ03 — Qual a taxa de falha das mudanças entregues?

Métrica: change failure rate (CFR), em duas variantes obrigatórias.

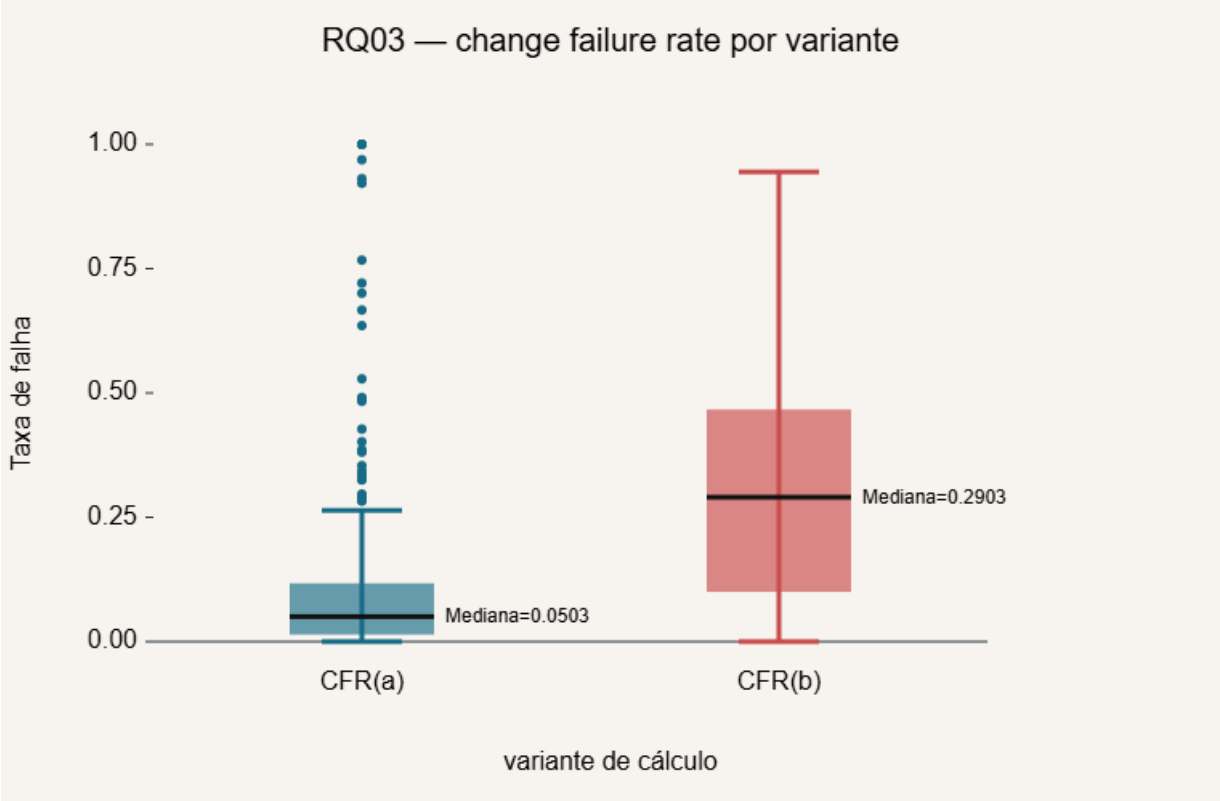


Figura 3 (Boxplot) — Comparação do Change Failure Rate entre as duas variantes.

O boxplot compara o CFR(a), baseado em falhas de workflows de CI, e o CFR(b), baseado na identificação de releases corretivas. O eixo x apresenta as duas variantes e o eixo y representa a taxa de falha, em uma escala de 0 a 1. A linha central de cada caixa representa a mediana, os limites da caixa representam o primeiro e o terceiro quartis e os pontos externos indicam possíveis valores extremos.

O CFR(a) apresentou mediana de 0,0503, equivalente a aproximadamente 5,03% de falhas de CI. O primeiro quartil foi 0,0144 e o terceiro quartil 0,1172, resultando em IQR de 0,1028. O CFR(b) apresentou mediana de 0,2903, equivalente a aproximadamente 29,03%. Para essa variante, o primeiro quartil foi 0,1000 e o terceiro quartil 0,4667, resultando em IQR de 0,3667. O cálculo do CFR(b) utilizou 299 repositórios válidos, pois um registro não possuía dados suficientes.

A diferença entre as medianas mostra que as duas variantes não medem exatamente o mesmo fenômeno. O CFR(a) representa falhas observadas no processo de integração contínua, enquanto o CFR(b) estima falhas de entrega a partir de releases corretivas identificadas por heurística. Por isso, o CFR(b) não deve ser interpretado como uma medida direta de falhas em produção.

4.2.4 RQ04 — Qual o tempo de recuperação após uma execução de CI/CD com falha?

Métrica: tempo de recuperação, em horas.

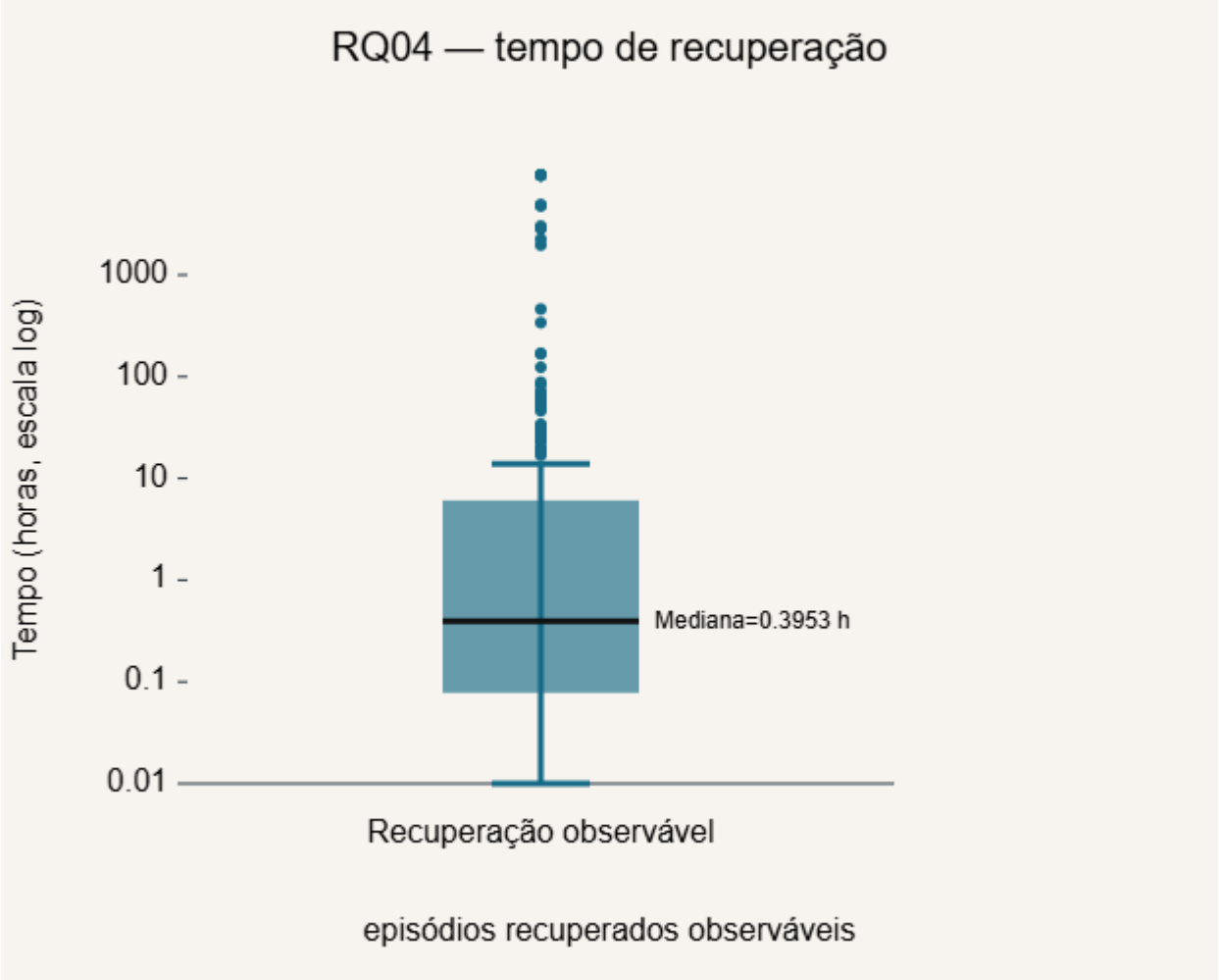


Figura 4 (Boxplot) — Distribuição do tempo de recuperação dos episódios observáveis.

O boxplot apresenta o tempo necessário para que um workflow com falha volte a apresentar uma execução bem-sucedida. O eixo x representa os episódios de recuperação observáveis e o eixo y apresenta o tempo de recuperação em horas, utilizando escala logarítmica para acomodar a assimetria e os valores extremos. A linha central indica a mediana, os limites da caixa representam o primeiro e o terceiro quartis e os pontos externos correspondem a possíveis valores extremos.

A mediana do tempo de recuperação foi de 0,3953 horas, aproximadamente 24 minutos. O primeiro quartil foi de 0,0778 horas e o terceiro quartil de 6,0644 horas, resultando em um IQR de 5,9867 horas. Isso indica que metade dos episódios observáveis foi recuperada entre aproximadamente cinco minutos e seis horas.

A visualização considera apenas os 261 repositórios que apresentaram episódios de recuperação observáveis. Os 39 casos restantes não possuíam uma mediana calculável, seja pela ausência de episódio, pela ausência de uma execução bem-sucedida posterior ou por censura no final da janela de observação. Esses casos não foram convertidos em zero, pois a ausência de recuperação observável não significa recuperação imediata.

4.2.5 RQ05 — Repositórios com maior frequência de deploy apresentam maior ou menor taxa de falha?

Métrica: Correlação de Spearman

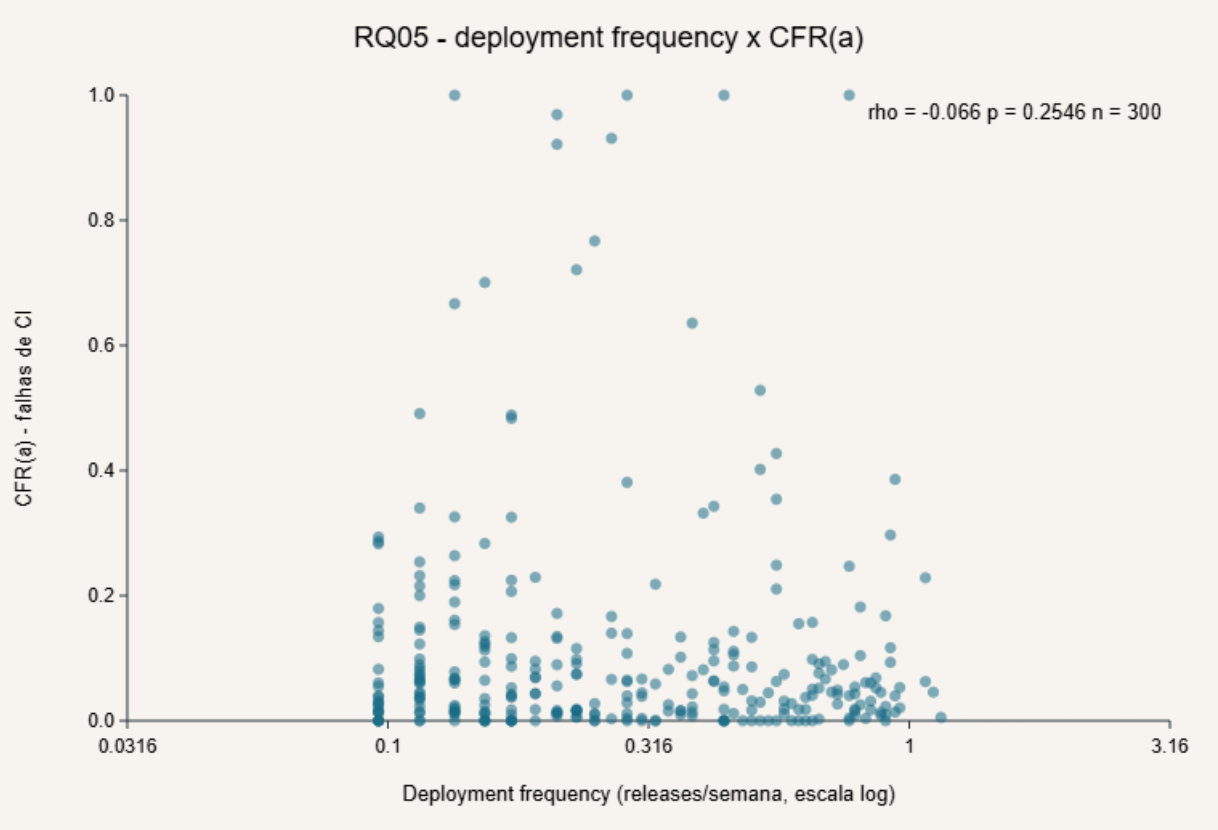


Figura 5 (Dispersão) — Relação entre deployment frequency e CFR(a).

O gráfico de dispersão apresenta a relação entre a frequência de releases e o CFR(a), calculado a partir de falhas de workflows de CI. O eixo x representa a deployment frequency em releases por semana, em escala logarítmica, e o eixo y representa o CFR(a), entre 0 e 1. Cada ponto corresponde a um repositório. A correlação de Spearman foi ρ = −0,066, com p = 0,2546 e n = 300, indicando uma associação negativa desprezível e sem significância estatística. Portanto, não há evidência de que repositórios com maior frequência de releases apresentem menor taxa de falhas de CI.

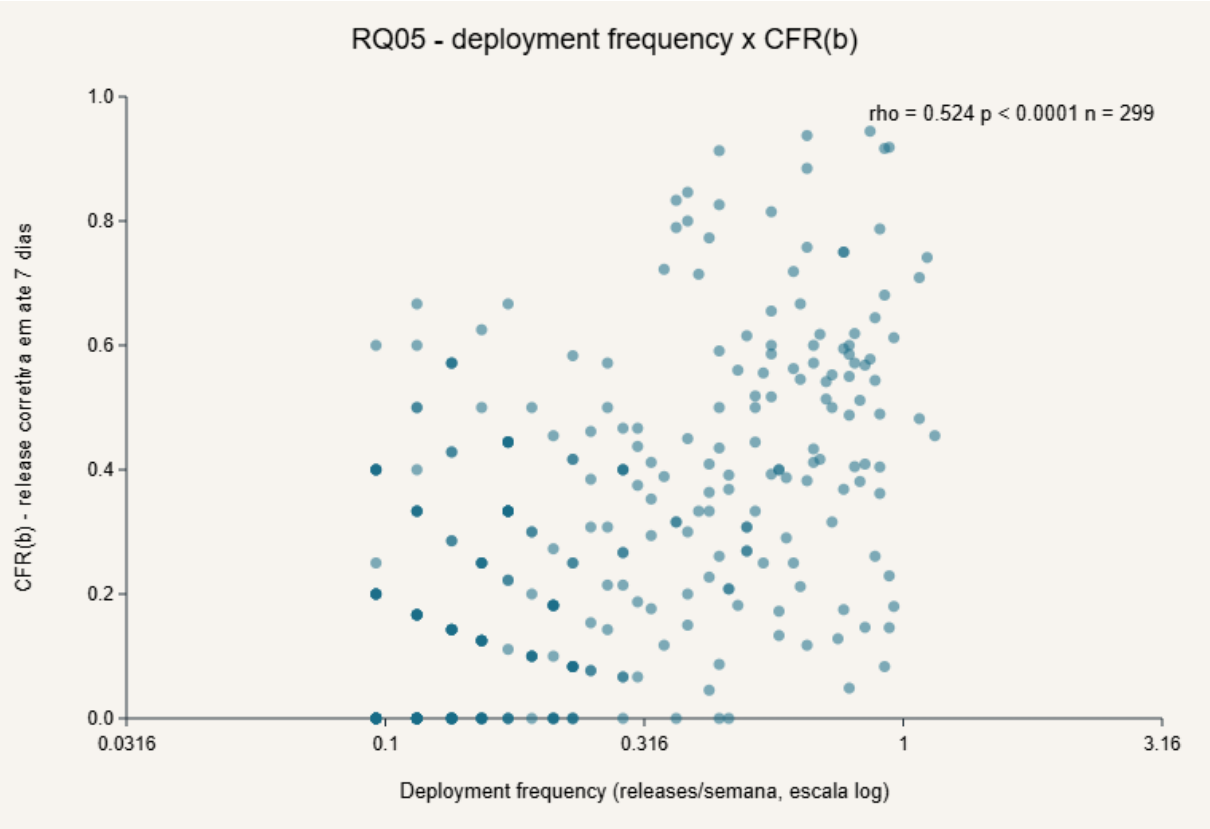


Figura 6 (Dispersão) — Relação entre deployment frequency e CFR(b).

O segundo gráfico utiliza o CFR(b), baseado na identificação de releases corretivas em até sete dias. A correlação de Spearman foi ρ = 0,524, com p < 0,0001 e n = 299, indicando uma associação positiva forte e estatisticamente significativa. Nesse caso, repositórios com maior frequência de releases tendem a apresentar CFR(b) maior. Entretanto, esse resultado deve ser interpretado com cautela, pois a própria janela de sete dias da heurística pode favorecer a identificação de releases corretivas em projetos que publicam com maior frequência. A correlação observada não permite concluir que a frequência de releases cause mais falhas.em zero, pois a ausência de recuperação observável não significa recuperação imediata.

4.2.6 RQ06 — Qual o tempo de recuperação após uma execução de CI/CD com falha?

Métrica: Comparação das métricas DORA (RQ 01 a 04) entre subgrupos de repositórios

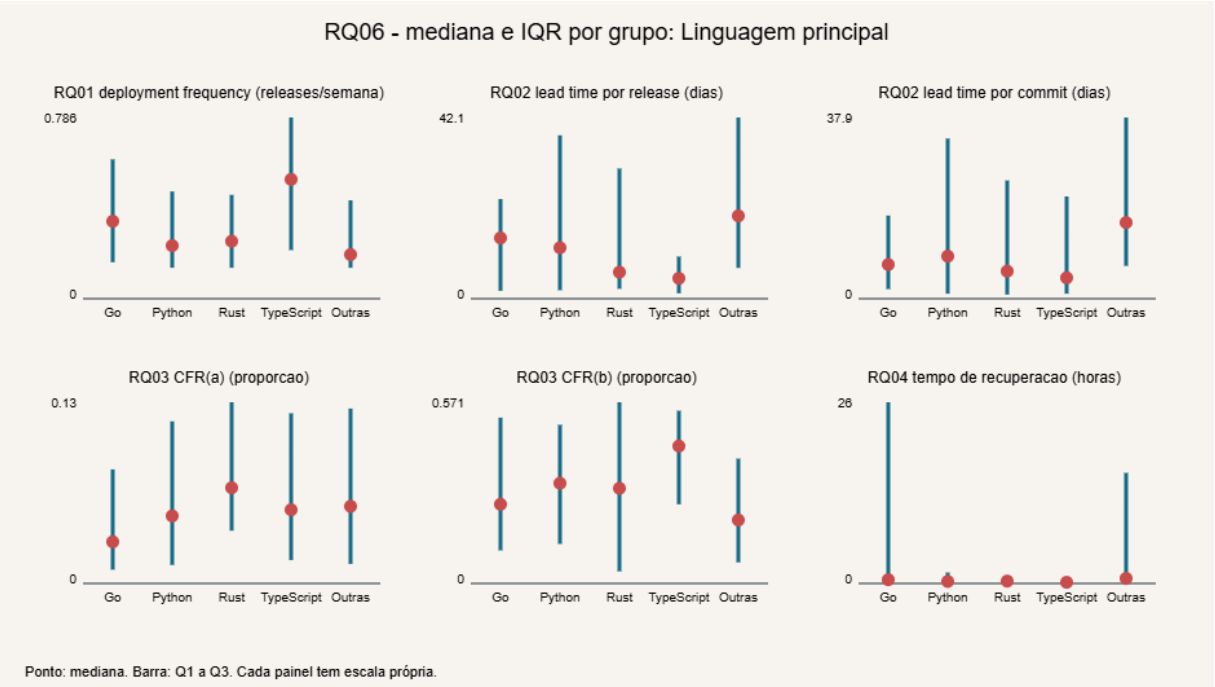


Figura 7 — Medianas e IQR das métricas DORA por linguagem principal.

A figura compara os grupos de linguagem principal, considerando apenas linguagens com representatividade suficiente e agrupando as demais em “Outras”. Em cada painel, o ponto representa a mediana da métrica e a barra vertical representa o intervalo interquartil. As diferenças mais relevantes entre linguagens ocorreram na deployment frequency, no lead time por release, no lead time por commit e no CFR(b). Os efeitos foram pequenos, com ε^2 de **0,078**, **0,078**, **0,057** e **0,066**, respectivamente.

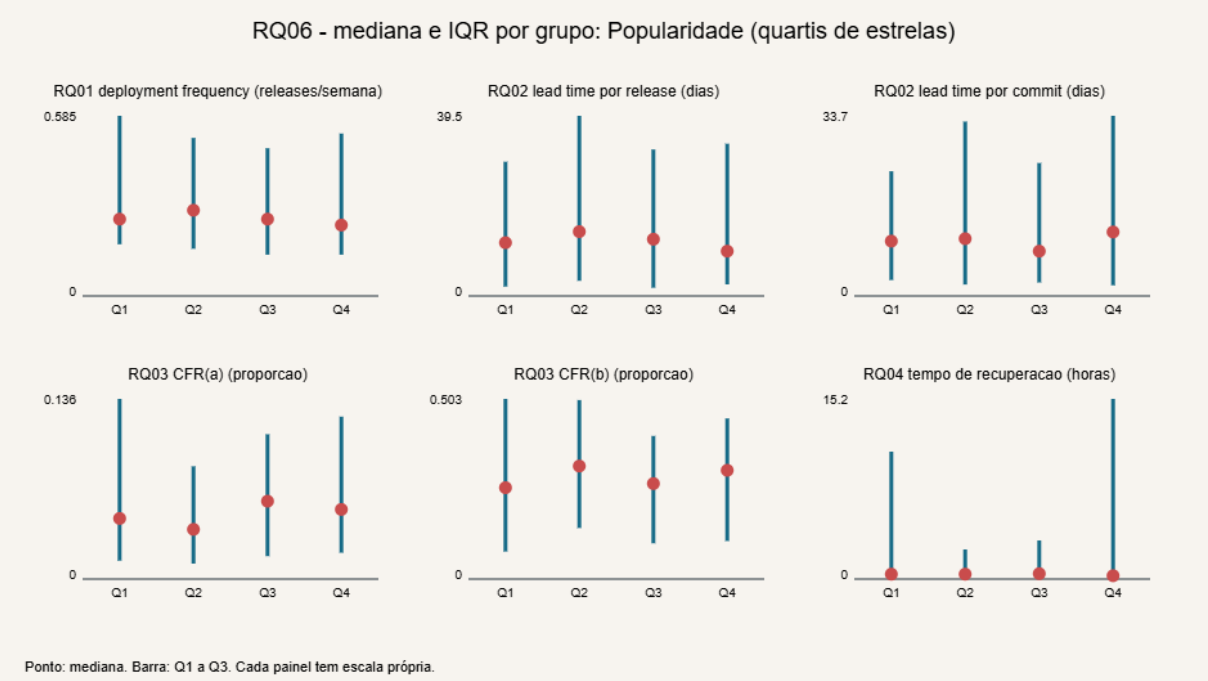


Figura 8— Medianas e IQR das métricas DORA por quartil de estrelas.

A figura divide os repositórios em quatro grupos de popularidade, definidos pelos quartis de estrelas. As medianas das métricas permaneceram próximas entre Q1, Q2, Q3 e Q4, sem diferenças

estatisticamente significativas após a correção de Holm. Os tamanhos de efeito foram desprezíveis, variando aproximadamente entre 0,001 e 0,009, indicando que a quantidade de estrelas não apresentou associação relevante com o desempenho DORA na amostra.

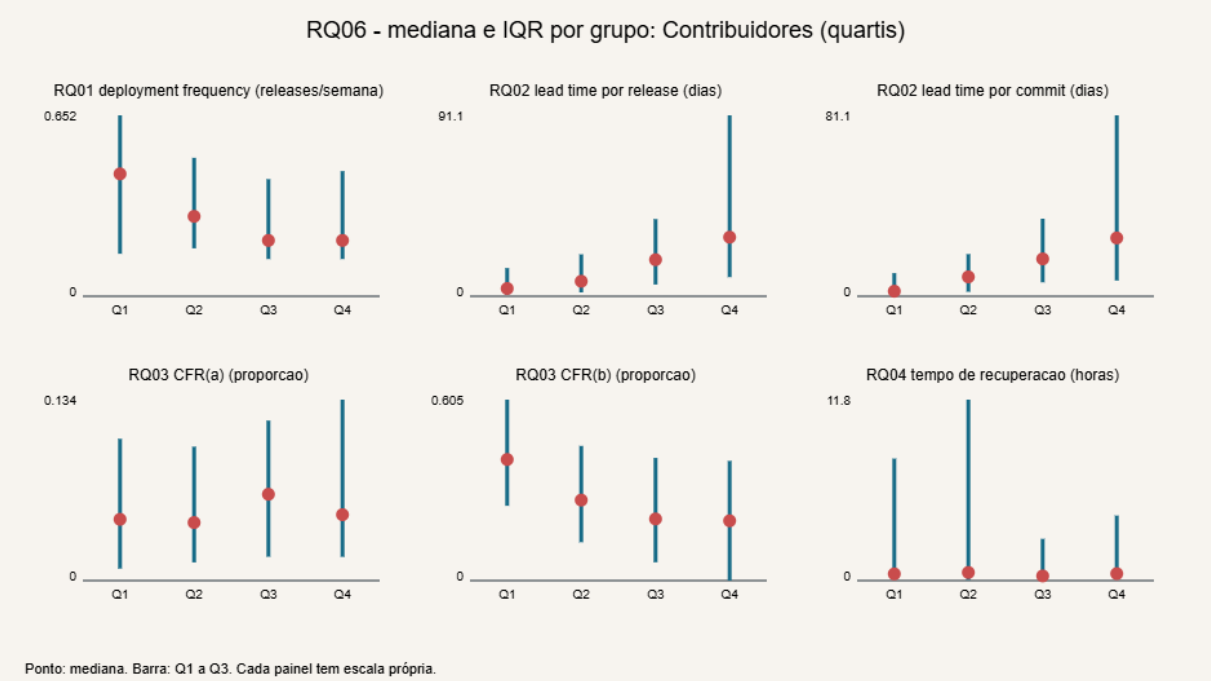


Figura 9— Medianas e IQR das métricas DORA por quartil de contribuidores.

A comparação por número de contribuidores apresentou diferenças mais expressivas. Os maiores efeitos ocorreram no lead time por release, com ε^2 de 0,189, e no lead time por commit, com ε^2 de 0,184, ambos classificados como efeitos médios. Também houve efeito significativo no CFR(b), embora de menor magnitude, com ε^2 de 0,076. A deployment frequency não permaneceu significativa após Holm, e o CFR(a) e o tempo de recuperação apresentaram efeitos desprezíveis.

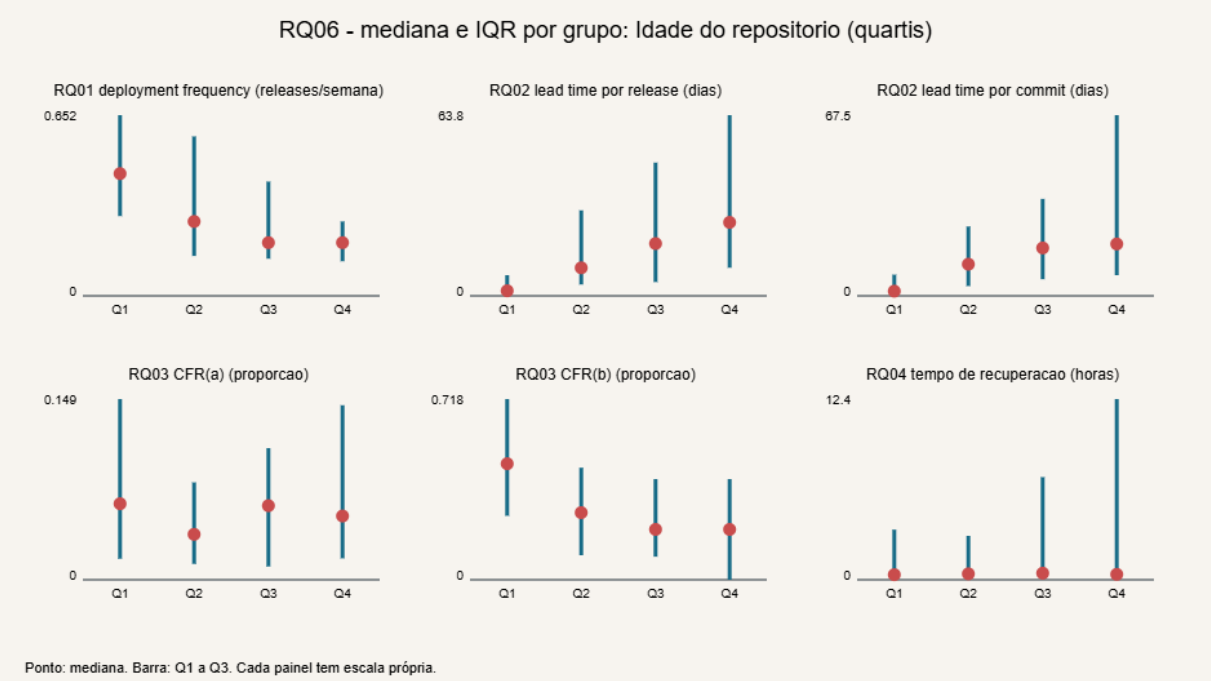


Figura 10 — Medianas e IQR das métricas DORA por quartil de idade do repositório

A idade do repositório foi o fator com diferenças mais consistentes. Foram observados efeitos significativos na deployment frequency, no lead time por release, no lead time por commit e no CFR(b). Os maiores tamanhos de efeito ocorreram nos dois tipos de lead time, com ε^2 de 0,236 e 0,240, respectivamente. Já o CFR(a) e o tempo de recuperação não apresentaram diferenças estatisticamente significativas entre os quartis.

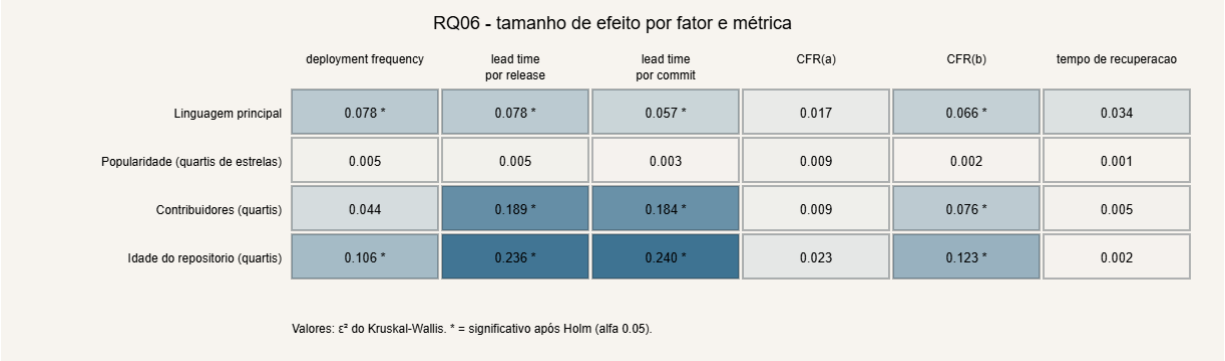


Figura 11 — Tamanhos de efeito por fator e métrica.

O heatmap resume os 24 testes realizados. Após a correção de Holm, 11 testes permaneceram significativos. A maior concentração de efeitos ocorreu nos fatores idade do repositório, número de contribuidores e linguagem principal. Em contraste, a popularidade por estrelas apresentou efeitos desprezíveis em todas as métricas. Os resultados indicam que características estruturais e históricas do projeto estão mais associadas ao desempenho DORA do que a popularidade medida isoladamente. Os valores apresentados representam associação entre os grupos e não permitem concluir causalidade.

4.2.7 RQ07 — Qual o tempo de recuperação após uma execução de CI/CD com falha?

Métrica: proporção de repositórios que mudam de categoria DORA entre definições, e concordância entre as classificações

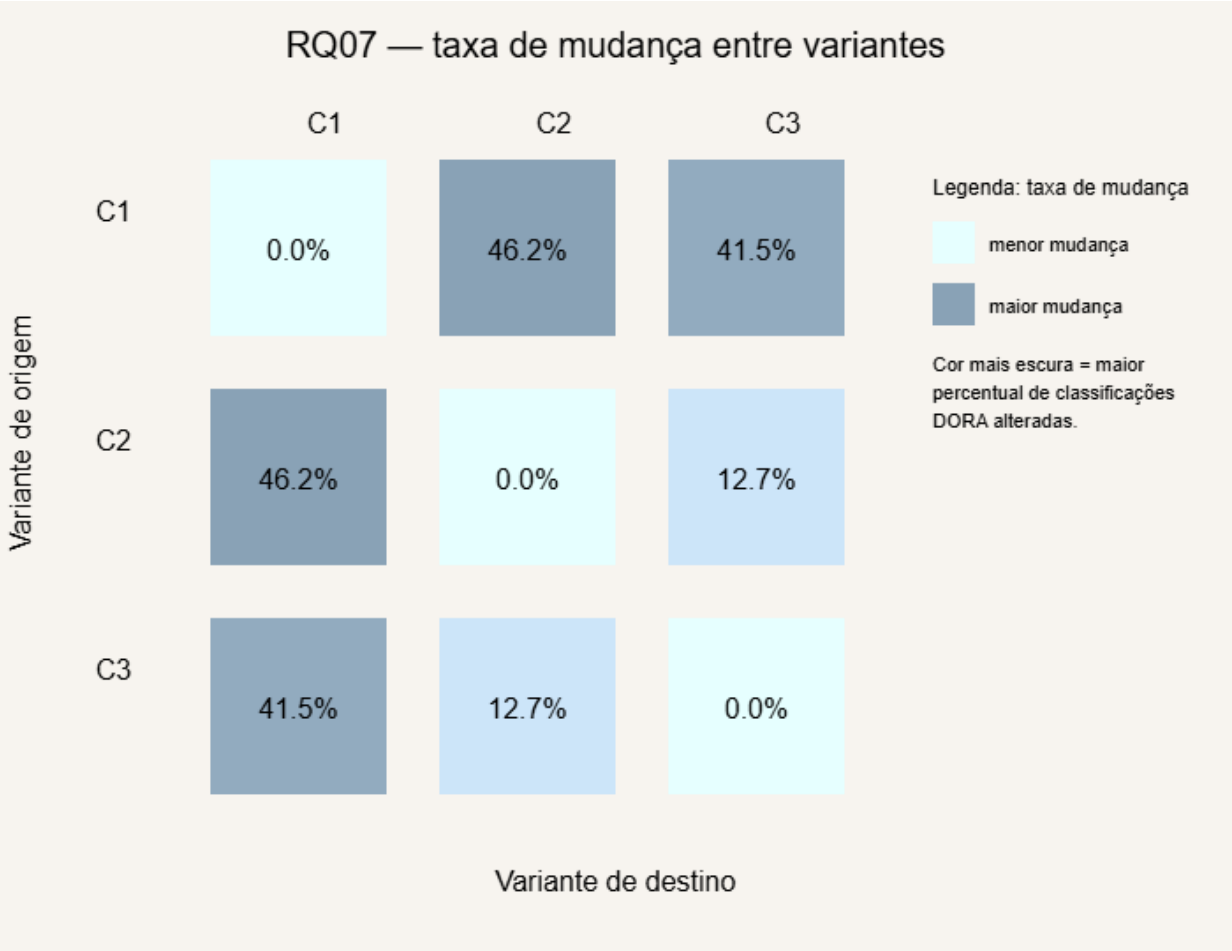


Figura 12 (Heatmap) — O quanto a classificação DORA depende da definição operacional escolhida?

O heatmap compara as classificações DORA produzidas pelas variantes C1, C2 e C3. O eixo x representa a Variante de destino e o eixo y representa a Variante de origem. Cada célula mostra o percentual de repositórios que mudou de categoria DORA entre as duas variantes. A legenda indica que cores mais escuras representam maior percentual de classificações alteradas. A diagonal principal apresenta valor igual a zero porque cada variante está sendo comparada consigo mesma.

A comparação entre C1 e C2 apresentou mudança em 46,1538% dos repositórios. Entre C1 e C3, a taxa de mudança foi de 41,5385%, enquanto entre C2 e C3 foi de 12,6923%. Esses valores mostram que a escolha da definição operacional pode alterar substancialmente a classificação final de um repositório.

As maiores mudanças ocorreram quando a variante de referência C1 foi comparada com variantes que alteravam o lead time ou o proxy de CFR. Já C2 e C3 apresentaram maior concordância entre si. Portanto, a classificação DORA não deve ser interpretada como independente das decisões metodológicas adotadas. A variante baseada em tags não foi incluída porque o dataset completo de tags não estava disponível; nenhum valor foi estimado para preencher essa ausência.

4.3 Discussão

A hipótese informal para a RQ01 era que repositórios populares com CI/CD apresentariam frequência elevada de releases, por possuírem maior maturidade e atividade de desenvolvimento. O resultado confirmou essa expectativa apenas parcialmente: a mediana foi de 0,2493 releases por semana, aproximadamente uma release a cada quatro semanas, com IQR de 0,4027. Portanto, a popularidade e o uso de CI/CD não implicam necessariamente entregas frequentes. Além disso, releases são apenas um proxy de deploy, pois a API do GitHub não permite confirmar diretamente se cada release chegou a produção.

Para a RQ02, a hipótese era que projetos ativos apresentariam baixo tempo entre a implementação de uma mudança e sua entrega. O resultado foi parcialmente confirmado: o lead time mediano foi de 11,6280 dias por release e 10,7128 dias por commit. As duas variantes apresentaram valores centrais próximos, indicando que a forma de agregação não alterou substancialmente o resultado geral. Entretanto, a dispersão elevada mostra que existem repositórios com ciclos de entrega muito mais longos. A diferença entre as variantes também reforça que um commit antigo incluído em uma release pode aumentar bastante o lead time calculado por release.

Na RQ03, a hipótese informal era que a taxa de falha seria baixa em repositórios populares e ativos. Essa hipótese foi parcialmente confirmada, mas a conclusão depende da definição utilizada. O CFR(a), baseado em falhas de CI, apresentou mediana de 0,0503, enquanto o CFR(b), baseado na heurística de releases corretivas, apresentou mediana de 0,2903. A diferença evidencia que falha de pipeline não é equivalente a falha de entrega ou falha em produção. Além disso, a heurística do CFR(b) apresentou F1 igual a 0,40 na validação manual, portanto seus resultados devem ser interpretados como uma estimativa aproximada, e não como observação direta de incidentes em produção.

A hipótese para a RQ04 era que repositórios com CI/CD conseguiriam recuperar-se rapidamente após falhas. O resultado foi favorável entre os casos observáveis: a mediana do tempo de recuperação foi de 0,3953 horas, aproximadamente 24 minutos. Porém, a métrica só foi calculável para 261 repositórios, enquanto 39 não apresentaram uma mediana observável. Esses casos podem representar ausência de episódios, ausência de sucesso posterior ou censura no final da janela. Assim, a hipótese foi confirmada apenas para os episódios recuperados, não podendo ser generalizada para todos os repositórios.

Na RQ05, a hipótese era que maior frequência de entrega não representaria necessariamente um trade-off com estabilidade, conforme a interpretação do modelo DORA. O resultado foi misto. Para o CFR(a), não houve associação estatisticamente significativa entre frequência e falha ($\rho = -0{,}066$; $p = 0{,}2546$), o que significa que os dados não permitem afirmar uma relação monotônica entre as duas variáveis nesse proxy. Para o CFR(b), foi observada uma associação positiva forte ($\rho = 0{,}524$; $p < 0{,}0001$): repositórios com mais releases também apresentaram maior proporção de releases corretivas segundo a heurística. Portanto, a hipótese foi parcialmente confirmada, mas o resultado do CFR(b) deve ser tratado com cautela, pois a janela de sete dias da heurística pode produzir uma associação mecânica com a própria frequência de releases. Além disso, correlação não implica causalidade.

A hipótese da RQ06 era que características dos repositórios estariam associadas a diferenças no desempenho DORA. Essa hipótese foi parcialmente confirmada. Após a correção de Holm, 11 dos 24 testes apresentaram resultados significativos. As diferenças mais consistentes ocorreram entre grupos definidos por linguagem, número de contribuidores e idade do repositório, principalmente para lead time e CFR(b). A popularidade medida por estrelas não apresentou efeito relevante. Como os tamanhos de efeito foram geralmente pequenos ou médios, significância estatística não significa necessariamente uma diferença operacional grande. Também é possível que os fatores estejam relacionados entre si,

gerando confundimento entre linguagem, idade, popularidade e número de contribuidores.

Na RQ07, a hipótese era que a classificação DORA seria relativamente robusta às escolhas operacionais. Essa hipótese foi refutada ou, no mínimo, fortemente enfraquecida. A comparação C1×C2 alterou 46,1538% das classificações, com kappa ponderado de 0,394997; C1×C3 alterou 41,5385%, com kappa de 0,453220; e C2×C3 alterou 12,6923%, com kappa de 0,828502. Em termos práticos, isso significa que a categoria DORA de muitos repositórios depende da definição de lead time e do proxy de CFR utilizado. A classificação não deve, portanto, ser apresentada como uma propriedade absoluta do repositório. A análise release-versus-tag não pôde ser concluída porque o dataset final não continha a variante baseada em tags; nenhum valor foi inferido ou simulado.

Os principais riscos à validade estão relacionados ao uso de proxies e à qualidade desigual dos dados. Releases não representam necessariamente deploys em produção, workflow runs não representam necessariamente incidentes reais e a heurística do CFR(b) apresentou desempenho limitado na amostra-ouro. A exigência de pelo menos cinco releases e 50 workflow runs também restringe a generalização dos resultados para repositórios populares e ativos, não representando todo o ecossistema open source. Além disso, dados ausentes e episódios censurados reduzem o tamanho efetivo de algumas análises. Nos subgrupos da RQ06, possíveis relações entre idade, linguagem, estrelas e contribuidores podem confundir as associações observadas.

As inovações do grupo aprofundaram os resultados exigidos pelo enunciado. O cache, a paginação e o tratamento de rate limit tornaram a coleta reproduzível e permitiram preservar as respostas brutas sem repetir desnecessariamente as consultas. A separação entre ausência de evidência e valor zero evitou conclusões artificialmente otimistas. Já a análise ampliada de sensibilidade mostrou que as conclusões DORA mudam conforme a definição operacional, complementando os resultados das RQs principais. Assim, as inovações não substituíram os resultados obrigatórios, mas revelaram suas condições de validade e tornaram mais explícitas as limitações da interpretação.

5. Conclusão

A análise mostrou que os repositórios populares com CI/CD não apresentam necessariamente alta frequência de releases: a cadência observada foi moderada e variou bastante entre os projetos. O lead time medido por release e por commit produziu resultados centrais semelhantes, mas com grande dispersão. A taxa de falha dependeu fortemente da definição utilizada: falhas de CI foram menos frequentes do que falhas inferidas por releases corretivas. Entre os episódios observáveis, a recuperação ocorreu rapidamente, embora parte dos repositórios não tenha apresentado episódios completos dentro da janela.

A relação entre velocidade e estabilidade não foi única. Não houve associação estatisticamente significativa entre frequência e CFR baseado em CI, mas houve associação positiva entre frequência e CFR baseado na heurística corretiva. As características dos repositórios também contribuíram para diferenças de desempenho: linguagem, idade e número de contribuidores apresentaram associações mais consistentes, enquanto popularidade por estrelas teve pouca influência. A análise de sensibilidade mostrou que a classificação DORA pode mudar de acordo com a definição de lead time e CFR, portanto uma classificação deve sempre informar suas escolhas operacionais.

As principais limitações são o uso de releases e workflows como proxies de deploy e falha em produção, a dependência da heurística do CFR(b), a ausência da variante completa baseada em tags, a censura de episódios no final da janela e a restrição da amostra a repositórios populares e ativos. O

estudo também cobre apenas o ano de 2025; por isso, não permite avaliar tendências históricas ou mudanças de comportamento ao longo de vários anos. Além disso, os subgrupos analisados podem conter fatores de confusão entre linguagem, idade, popularidade e número de contribuidores.

Como recomendação prática, equipes que desejem utilizar métricas DORA devem registrar releases e deploys de forma consistente, manter workflows com conclusões confiáveis, monitorar separadamente falhas de pipeline e incidentes de produção e documentar o tempo de recuperação. Também devem evitar comparar classificações DORA produzidas com definições diferentes sem realizar uma análise de sensibilidade. O cache, os manifestos e os datasets versionados desenvolvidos pelo grupo podem ser utilizados para repetir a coleta e acompanhar a evolução das métricas.

Com mais tempo e recursos, o grupo ampliaria a coleta de tags para todos os repositórios, incluiria dados reais de produção e acompanharia a amostra por vários anos. Também seria importante aumentar a amostra-ouro, melhorar a heurística de releases corretivas com modelos mais precisos e controlar estatisticamente os fatores de confusão entre os subgrupos. Entre as inovações propostas, a arquitetura reproduzível de coleta, o tratamento explícito de dados ausentes e a análise de sensibilidade são as contribuições mais promissoras para trabalhos futuros, pois podem ser aplicadas a diferentes amostras e laboratórios sem alterar o princípio da metodologia.

.

6. Referências

- DORA. DORA Metrics: The Four Keys. Disponível em: https://dora.dev/guides/dora-metrics-four-keys/. Acesso em: 20 out. 2026.
- FORSgren, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.
- GITHUB. GitHub REST API Documentation. Disponível em: https://docs.github.com/en/rest. Acesso em: 20 out. 2026.
- GITHUB. REST API endpoints for releases. Disponível em: https://docs.github.com/en/rest/releases/releases. Acesso em: 20 out. 2026.
- GITHUB. REST API endpoints for workflow runs. Disponível em: https://docs.github.com/en/rest/actions/workflow-runs. Acesso em: 20 out. 2026.
- GITHUB. REST API endpoints for comparing commits. Disponível em: https://docs.github.com/en/rest/commits/commits#compare-two-commits. Acesso em: 20 out. 2026.
- GITHUB. Using pagination in the REST API. Disponível em: https://docs.github.com/en/rest/using-the-rest-api/using-pagination-in-the-rest-api. Acesso em: 20 out. 2026.
- GITHUB. Rate limits for the REST API. Disponível em: https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api. Acesso em: 20 out. 2026.
- FLEISS, Joseph L. Measuring nominal scale agreement among many raters. Psychological Bulletin, v. 76, n. 5, p. 378–382, 1971. DOI: 10.1037/h0031619.
- COHEN, Jacob. A coefficient of agreement for nominal scales. Educational and Psychological Measurement, v. 20, n. 1, p. 37–46, 1960. DOI: 10.1177/001316446002000104.

- CLIFF, Norman. Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological Bulletin, v. 114, n. 3, p. 494–509, 1993. DOI: 10.1037/0033-2909.114.3.494.
- NUMPY; SCIPY. SciPy Statistical Functions Documentation. Disponível em: https://docs.scipy.org/doc/scipy/reference/stats.html. Acesso em: 20 out. 2026.
- STATSMODELS. Multiple Testing and Multipletests Documentation. Disponível em: https://www.statsmodels.org/stable/generated/statsmodels.stats.multitest.multipletests.html. Acesso em: 20 out. 2026.
- SEMVER. Semantic Versioning 2.0.0. Disponível em: https://semver.org/lang/pt-BR/. Acesso em: 20 out. 2026.